Convergencia Arquitectónica: Ingeniería de Skills para Google Antigravity en Ecosistemas de Producción Unity 3D

Resumen Ejecutivo

La industria del desarrollo de software se encuentra en un punto de inflexión crítico, transitando desde un modelo centrado en el desarrollador —donde los ingenieros orquestan manualmente herramientas dispares— hacia un paradigma "Agent-First" (centrado en el agente), donde sistemas de Inteligencia Artificial autónomos planifican, ejecutan y verifican flujos de trabajo complejos con mínima intervención humana. Google Antigravity emerge como la implementación insignia de esta arquitectura, introduciendo un Entorno de Desarrollo Integrado (IDE) diseñado no meramente para el autocompletado de código, sino para la orquestación de tareas de alto nivel a través de modelos de razonamiento avanzados como Gemini 3 Pro.[1]

Este informe técnico proporciona un análisis exhaustivo ("Deep Research") sobre la arquitectura de la plataforma Antigravity, con un enfoque quirúrgico en su mecanismo de extensibilidad principal: las "Skills" (Habilidades) y el Protocolo de Contexto de Modelo (MCP). El documento aborda la brecha existente entre la ingeniería de software agéntica de propósito general y el dominio especializado y altamente idiosincrásico del desarrollo de videojuegos AAA con Unity 3D. A través de la síntesis de datos técnicos sobre el Data-Oriented Technology Stack (DOTS) de Unity, la automatización vía Interfaz de Línea de Comandos (CLI) y la definición estructurada de Skills en formato SKILL.md, se delinea un marco arquitectónico robusto para la automatización de pipelines de producción. La convergencia de las capacidades de razonamiento de Antigravity con la arquitectura ECS de Unity ofrece una ruta viable para reducir la carga cognitiva en los ingenieros de motores, automatizar pruebas de integración tediosas y forzar la adherencia a patrones arquitectónicos estrictos a escala masiva.

---

1. El Paradigma Agent-First: Deconstrucción de la Arquitectura Google Antigravity

Para abordar la creación efectiva de Skills, es imperativo comprender primero el entorno de ejecución subyacente. Google Antigravity se distingue radicalmente de los IDEs tradicionales (como Visual Studio o Rider) e incluso de editores asistidos por IA (como Cursor o Windsurf) al tratar a la Inteligencia Artificial no como un asistente pasivo, sino como un actor autónomo con capacidad de agencia.[1] Esta distinción no es semántica, sino arquitectónica, y define

cómo se deben estructurar las interacciones y las herramientas.

1.1 Superficies de Interacción y Orquestación

La plataforma se construye sobre tres superficies de interacción distintas, cada una diseñada para soportar una fase específica del flujo de trabajo agéntico. Comprender estas superficies es crucial para el diseño de Skills, ya que determina dónde y cómo se presentará la información al usuario y al modelo.

En primer lugar, la **Vista del Editor** mantiene la familiaridad de un editor de código estándar, equipado con funcionalidades como IntelliSense y resaltado de sintaxis. Sin embargo, su propósito subyacente en Antigravity es servir como el "lienzo" sobre el cual actúa el agente. A diferencia de los entornos anteriores donde la IA sugería cambios que el usuario debía aceptar, en Antigravity el editor es una herramienta que el agente manipula directamente. Esto implica que las Skills deben diseñarse teniendo en cuenta que el agente "lee" y "escribe" en este buffer de manera autónoma, utilizando herramientas de manipulación de archivos y análisis estático disponibles en el entorno.[1]

En segundo lugar, y representando la desviación más significativa, se encuentra el **Gestor de Agentes (Agent Manager)**. Esta superficie actúa como una interfaz de "Control de Misión". Está diseñada para la orquestación de alto nivel, permitiendo a los desarrolladores instanciar, monitorear e interactuar con múltiples agentes que operan de manera asíncrona. Esta capacidad de paralelismo es fundamental: un agente puede estar refactorizando un sistema de físicas en C# mientras otro ejecuta scripts de compilación en segundo plano y un tercero analiza logs de errores. El desacoplamiento del foco inmediato del usuario respecto al hilo de ejecución del agente es lo que permite escalar la productividad más allá de la velocidad de escritura humana.[1]

Finalmente, el **Agente de Navegador** proporciona una instancia de navegador (headless o visible) que los agentes pueden actuar. Esto habilita flujos de trabajo de verificación "end-to-end" (E2E). En el contexto de Unity, aunque Unity no es una aplicación web, esta capacidad es vital para interactuar con dashboards de DevOps, documentación en línea, o consolas de administración de servidores en la nube (como Unity Gaming Services), permitiendo al agente verificar despliegues o consultar documentación API en tiempo real sin alucinar.[5]

1.2 La Función Crítica de los Artefactos en la Calibración de Confianza

Uno de los desafíos recurrentes en los sistemas agénticos es el problema de la "caja negra": los usuarios desconfían de un sistema cuyo proceso de razonamiento es opaco. Antigravity aborda esto mediante la generación de **Artefactos**: entregables tangibles producidos durante la operación del agente.[3]

Los artefactos no son simples logs de texto; son objetos estructurados que sirven como puntos de control para la verificación humana. Incluyen:

- **Listas de Tareas y Planes de Implementación:** Generados antes de escribir una sola línea de código, permitiendo al arquitecto humano corregir la lógica del agente a nivel de diseño.

- **Diffs de Código:** Cambios granulares presentados para revisión antes de su aplicación definitiva.
- **Informes de Pruebas y Capturas de Pantalla:** Evidencia visual de funcionalidad.

La implicación arquitectónica para el diseño de Skills es profunda: una Skill de alta calidad no debe limitarse a ejecutar un script silenciosamente. Debe estar diseñada para producir una salida que Antigravity pueda renderizar como un Artefacto. Por ejemplo, una Skill de "Build de Unity" no solo debe invocar el compilador, sino que debe parsear el Editor.log y generar un objeto JSON que el UI de Antigravity pueda mostrar como un "Informe de Errores de Compilación", permitiendo al usuario aprobar correcciones o detener el proceso.[1]

1.3 Saturación de Contexto y Divulgación Progresiva

Los Modelos de Lenguaje Grande (LLMs) modernos, como Gemini 3 Pro o Claude 3.5 Sonnet, poseen ventanas de contexto masivas (superando los 2 millones de tokens). Sin embargo, llenar estas ventanas con datos irrelevantes conduce inevitablemente a la "Saturación de Contexto" (Context Saturation), un estado donde la capacidad de razonamiento del modelo se degrada debido a la relación señal/ruido.[7]

Antigravity mitiga este riesgo mediante una estrategia de **Divulgación Progresiva (Progressive Disclosure)**. A diferencia de los "System Prompts" tradicionales que se cargan permanentemente al inicio de la sesión, Antigravity utiliza un enrutador semántico de alto nivel. Inicialmente, el modelo solo tiene acceso a los metadatos ligeros de las Skills disponibles (definidos en el YAML frontmatter). El contenido "pesado" de una Skill —sus instrucciones detalladas, cuerpos de scripts, plantillas de código y documentación de referencia— solo se inyecta en la ventana de contexto activa cuando el agente determina, mediante coincidencia semántica, que la Skill es relevante para la solicitud actual del usuario.[8]

Este mecanismo es vital para proyectos de videojuegos AAA, que pueden contener cientos de herramientas especializadas. Permite tener una biblioteca de Skills masiva (e.g., "Generar Boilerplate DOTS", "Parsear Logs de Build", "Optimizar Asset Bundles", "Validar Naming Conventions") sin abrumar al agente ni incurrir en costos prohibitivos de tokens o latencia durante conversaciones generales.

---

2. Ingeniería de Skills en Antigravity: El Estándar SKILL.md

La unidad fundamental de extensibilidad en Antigravity es la Skill. Es crucial entender que una Skill no es simplemente un prompt guardado; es un paquete estructurado de conocimiento procedimental y herramientas ejecutables.

2.1 Jerarquía de Directorios y Alcance (Scope)

Las Skills se organizan en directorios físicos, lo que asegura su portabilidad y facilita el control de versiones (Git). Antigravity reconoce dos alcances distintos para la definición de Skills, permitiendo una granularidad en la personalización del entorno:

- **Alcance de Espacio de Trabajo (Workspace Scope):** Ubicadas en <workspace-root>/.agent/skills/. Estas Skills son específicas del proyecto actual. En un entorno de Unity, aquí residirían scripts para construir AssetBundles específicos del juego, herramientas para validar la estructura de la escena, o generadores de código para sistemas propietarios del estudio. Estas Skills viajan con el repositorio, asegurando que cualquier desarrollador (o agente) que clone el proyecto tenga acceso a las mismas herramientas.[9]
- **Alcance Global (Global Scope):** Ubicadas en ~/.gemini/antigravity/skills/ (o rutas de usuario equivalentes). Estas Skills están disponibles transversalmente en todos los proyectos del usuario. Son ideales para utilidades generales, como formateadores de C#, analizadores de logs genéricos, o herramientas de gestión de Git.[10]

La estructura de directorio recomendada para una Skill de producción ("Production-Grade") separa claramente la lógica, la instrucción y el conocimiento estático:

```
my-unity-skill/
├── SKILL.md # El archivo de definición (Cerebro: Metadatos + Instrucciones)
├── scripts/ # Lógica ejecutable (Músculo: Python, Bash, Node)
│ ├── build.py # Wrapper para Unity CLI
│ └── parser.py # Lógica de regex para logs
├── references/ # Conocimiento estático (Contexto: API docs, Plantillas)
│ ├── ecs-best-practices.md
│ └── architecture-diagram.mermaid
└── assets/ # Media de soporte
```

Esta separación de preocupaciones [7] es crítica. Permite que el agente lea las instrucciones en SKILL.md para entender *qué* hacer, mientras que la complejidad de la ejecución se delega a los scripts en scripts/, manteniendo la ventana de contexto limpia de código imperativo innecesario.

2.2 El Archivo de Definición SKILL.md

El archivo SKILL.md actúa como la interfaz entre el motor de razonamiento del LLM y las herramientas subyacentes. Se compone de dos secciones obligatorias: el Frontmatter YAML y el Cuerpo Markdown.[12]

2.2.1 YAML Frontmatter: La Capa de Enrutamiento

El frontmatter es la única parte de la Skill visible para el agente durante la fase de descubrimiento. Su redacción requiere precisión técnica, ya que actúa como el "trigger" funcional.

- **name (Requerido):** Identificador único, típicamente en kebab-case (e.g., unity-build-automation).
- **description (Requerido):** Este campo es el más crítico. No es una simple descripción para humanos; es el embedding semántico que el modelo utiliza para decidir si "equipa" la Skill. Una descripción vaga ("Ayuda con Unity") resultará en una baja tasa de activación. Una descripción óptima debe articular la capacidad funcional y el contexto

de uso: *"Executes a headless Unity build for the target platform (Windows/Android/iOS), parses the Editor.log for compilation errors, and reports the build status. Use this when the user asks to 'build the game' or 'check for compile errors'."*.[10]

2.2.2 Markdown Body: La Capa de Instrucción

Una vez que la Skill es activada, el contenido Markdown se inyecta en el contexto. Esto es, en esencia, "Prompt Engineering persistido en archivo".[10]
Los componentes clave de un cuerpo efectivo incluyen:

1. **Objetivo (Goal):** Una declaración concisa del resultado esperado.
2. **Reglas y Restricciones (Constraints):** Límites duros sobre el comportamiento del agente. En Unity, esto es vital: *"Do not modify ProjectSettings files manually," "Always use Debug.Log instead of Console.WriteLine," "Use SerializeField instead of public fields."*.[10]
3. **Procedimiento (Procedure):** Flujo lógico paso a paso. Aquí es donde se instruye al agente para invocar las herramientas: *"1. Run the build script. 2. If exit code is 0, verify the artifact exists. 3. If exit code is 1, read the error log."*
4. **Ejemplos (Few-Shot Examples):** Pares de entrada/salida que demuestran el formato esperado de la respuesta o el código generado, guiando al modelo hacia el estilo deseado.[14]

2.3 Lógica Ejecutable: Integración de Python como Lenguaje "Pegamento"

Aunque el LLM proporciona el razonamiento, la manipulación efectiva del sistema operativo se realiza a través de herramientas. Antigravity expone una herramienta nativa run_command que las Skills pueden solicitar ejecutar.[7]
Para el desarrollo en Unity, **Python** se establece como el lenguaje "pegamento" superior frente a Bash o Batch. Unity genera logs complejos y no estructurados y requiere una gestión de procesos delicada (tiempos de espera, procesos colgados). Python ofrece bibliotecas robustas (subprocess, re, json) que permiten encapsular esta complejidad. Una Skill típica instruiría al agente de la siguiente manera:
"To build the project, execute python scripts/build.py --target Android using the run_command tool. Analyze the JSON output to determine if the build succeeded."
Esta delegación crea una separación de responsabilidades: el Agente maneja la intención del usuario y la recuperación de errores de alto nivel, mientras que el script de Python maneja la ejecución determinista del binario de Unity y la limpieza de datos.[15]

---

3. Automatización de la Interfaz de Unity 3D: El Puente Técnico

Para aplicar las Skills de Antigravity de manera efectiva en Unity, es necesario dominar los mecanismos mediante los cuales el motor puede ser controlado programáticamente

"headless" (sin interfaz gráfica). Unity es, por diseño, una aplicación GUI, lo que presenta desafíos únicos para la automatización.

3.1 La Interfaz de Línea de Comandos (CLI) del Editor

El Editor de Unity puede ser invocado desde la línea de comandos, permitiendo su control por scripts externos (y por ende, por agentes de Antigravity). Los argumentos críticos para la automatización fiable son:

- **-batchmode**: Ejecuta Unity sin ventana. Es esencial para agentes que operan en segundo plano para evitar robar el foco del usuario o para operar en servidores de integración continua (CI) sin GPU.[17]
- **-quit**: Fuerza al Editor a cerrarse una vez que el método ejecutado se completa. Sin este argumento, el agente quedaría colgado indefinidamente esperando la terminación del proceso.[19]
- **-executeMethod <Namespace.Class.Method>**: El punto de entrada principal. Instruye a Unity para ejecutar un método estático de C# específico inmediatamente después del inicio.[18]
- **-logFile <path>**: Redirige la salida del Editor. Por defecto, Unity escribe en un archivo oculto (Editor.log o Player.log). Para la automatización, es crucial redirigir esto explícitamente a stdout (usando -logFile - en algunas configuraciones) o a una ruta conocida y accesible por el script de Python para su análisis en tiempo real.[19]

3.2 El Desafío del Logging y la Solución Python

Un obstáculo significativo en la automatización de Unity es que la salida de la CLI es extremadamente verbosa y carece de estructura (no es JSON ni XML nativo). Un agente de Antigravity que intentara leer el Editor.log en crudo encontraría rápidamente una saturación de contexto debido al ruido de los mensajes de importación de assets y depuración irrelevante.

La Solución Arquitectónica: Se requiere un script "wrapper" en Python embebido en la carpeta scripts/ de la Skill. Este script realiza las siguientes funciones críticas:

1. **Gestión de Subprocesos:** Lanza Unity como un subprocess y gestiona su ciclo de vida.
2. **Streaming de Logs:** Lee el flujo de salida (stdout/stderr) o hace "tail" al archivo de log en tiempo real.[22]
3. **Filtrado y Parsing:** Utiliza expresiones regulares (Regex) para identificar y extraer únicamente la información relevante: Errores de Compilación (CSXXXX), Excepciones de Runtime y Reportes de Build (BuildReport).
4. **Estructuración de Datos:** Agrega estos datos en un objeto JSON estructurado.
5. **Comunicación con el Agente:** Imprime el JSON final en stdout para que la herramienta run_command de Antigravity lo capture y lo pase al LLM.

Este enfoque transforma 10,000 líneas de ruido de log en un resumen JSON conciso y accionable: {"status": "failed", "errors":}.[16]

---

4. Implementación Arquitectónica: Skills para Unity DOTS y Clean Architecture

La transición de Unity hacia el Data-Oriented Technology Stack (DOTS) y el Entity Component System (ECS) representa un cambio paradigmático en el estilo de codificación. DOTS exige una adherencia estricta a diseños de memoria específicos y el uso de atributos de compilación (``) para lograr las ganancias de rendimiento prometidas.[24] Esta rigurosidad sintáctica lo convierte en un candidato ideal para la generación de código asistida por IA mediante Skills.

4.1 Escenario de Skill: Generación de ISystems Compatibles con Burst

Escribir código ECS altamente optimizado implica una cantidad significativa de "boilerplate" propenso a errores humanos. Una Skill de Antigravity denominada dots-system-generator puede ser diseñada para hacer cumplir las mejores prácticas de 2025.

Configuración de SKILL.md:

- **Trigger:** "Create a new ECS System", "Optimize this loop with Burst."
- **Contexto:** La Skill carga documentación de referencia sobre ISystem, SystemAPI y la ubicación correcta del atributo ``.

Restricciones Técnicas Aplicadas por la Skill:

1. **Preferencia por ISystem:** A partir de 2025, ISystem (structs no manejados) es preferido sobre SystemBase (clases manejadas) para sistemas compatibles con Burst. La Skill debe forzar definiciones partial struct.[26]
2. **Ubicación de BurstCompile:** La Skill debe "saber" que el atributo `` debe colocarse en los métodos OnUpdate, OnCreate y OnDestroy, pero *no* necesariamente en el struct mismo en las versiones más recientes del paquete Entities, o estrictamente en el struct en versiones anteriores. La Skill maneja este matiz de versionado.[28]
3. **Generación de Queries Seguras:** La Skill genera automáticamente bucles SystemAPI.Query<RefRW<T>>() e IJobEntity, asegurando el uso correcto de acceso de Lectura/Escritura (RefRW) versus Solo Lectura (RefRO). Esto es fundamental para facilitar la seguridad en la programación de trabajos (Jobs) y evitar condiciones de carrera.[28]

Ejemplo de Artefacto Generado (Snippet C#):

C#

```
using Unity.Entities;
using Unity.Burst;
using Unity.Transforms;
```

```
public partial struct SpawnerSystem : ISystem {

   public void OnCreate(ref SystemState state) {
      state.RequireForUpdate<Spawner>();
   }


   public void OnUpdate(ref SystemState state) {
      foreach (var (transform, spawner) in SystemAPI.Query<RefRW<LocalTransform>,
RefRO<Spawner>>()) {
         // Logic inserted by agent based on intent
      }
   }
}
```

Este nivel de automatización reduce la carga cognitiva del desarrollador, permitiéndole enfocarse en la *lógica* de la simulación en lugar de luchar con la *sintaxis* del compilador Burst.[25]

4.2 Inyección de Dependencias con VContainer y Clean Architecture

Para las partes de la arquitectura que no utilizan DOTS (Game Objects tradicionales, UI, Servicios de Red), los principios de "Clean Architecture" son cada vez más adoptados para desacoplar la lógica de negocio de la API de Unity. VContainer es una biblioteca de Inyección de Dependencias (DI) de alto rendimiento utilizada para este fin.[30]
Una Skill de Antigravity puede gestionar la complejidad de los "Installers". En VContainer, las dependencias se registran en un LifetimeScope. A medida que el proyecto crece, este código de registro puede volverse caótico y propenso a errores de tiempo de ejecución.
Lógica de la Skill vcontainer-installer-manager:

1. **Análisis:** El agente escanea la base de código buscando clases que implementen interfaces de servicio (e.g., IAudioService, INetworkClient) o que estén marcadas para inyección.
2. **Planificación:** Identifica qué servicios faltan en el GameLifetimeScope o SceneLifetimeScope.
3. **Ejecución:** Modifica el método Configure(IContainerBuilder builder) para registrar estos servicios con el tiempo de vida correcto (Singleton vs. Scoped vs. Transient), generando el código de registro apropiado (builder.Register<T>(Lifetime.Singleton)).[32]

Esta Skill previene las temidas NullReferenceException en tiempo de ejecución causadas por bindings faltantes, un problema endémico en proyectos Unity con alta dependencia de DI.[34]

---

5. Integración Profunda: Protocolo de Contexto de

Modelo (MCP)

Mientras que las Skills definen *cómo* procesar la información y ejecutar tareas, el Protocolo de Contexto de Modelo (MCP) define *cómo* acceder a esa información de manera segura y estandarizada.[35]

5.1 Diferenciación Crítica: Skills vs. MCP en Unity

Es fundamental distinguir entre ambos conceptos en la arquitectura de Antigravity:

- **Skill:** Es el flujo de trabajo o la metodología ("Realiza una revisión de código en el PlayerController siguiendo las guías de estilo").
- **Servidor MCP:** Es la capacidad de acceso a datos ("Lee el contenido actual de Assets/Scripts/PlayerController.cs" o "Dame la lista de GameObjects en la escena actual").

Antigravity utiliza MCP para conectarse al sistema de archivos local por defecto. Sin embargo, el verdadero poder se desbloquea al crear un **Servidor MCP de Unity personalizado**. Este servidor, ejecutándose dentro del proceso del Editor de Unity (mediante un socket C#), puede exponer el estado profundo del motor como recursos consultables.[37]

5.2 Implementación de un Servidor MCP para Unity

Un servidor MCP implementa el protocolo JSON-RPC 2.0. En el contexto de Unity, se puede crear un script de Editor ([InitializeOnLoad]) que levante un servidor ligero.

Flujo de Trabajo Hipotético con MCP de Unity:

1. **Consulta del Usuario:** "¿Hay algún GameObject en la escena actual que tenga referencias de script perdidas?"
2. **Activación de Skill:** El enrutador activa la Skill scene-auditor.
3. **Llamada a Herramienta MCP:** La Skill invoca la herramienta get_scene_hierarchy() expuesta por el servidor MCP de Unity.
4. **Retorno de Datos:** El servidor MCP recorre la jerarquía de la escena en Unity, serializa el árbol de objetos a JSON y lo devuelve al agente.
5. **Lógica del Agente:** El agente itera sobre el JSON, buscando nodos con propiedades script: null o componentes "Missing".
6. **Respuesta:** El agente reporta los nombres de los GameObjects afectados.

Esta integración transforma el IDE de un simple editor de texto a un asistente "Consciente de la Escena" (Scene Aware), capaz de razonar sobre la estructura del juego en tiempo de ejecución, no solo sobre el código fuente estático.[35]

---

6. Validación Automatizada: Test Runner y Métricas de Rendimiento

El desarrollo de juegos fiable requiere pruebas continuas. Unity dispone de un Test Runner robusto, pero su integración en flujos de trabajo automatizados a menudo es manual.

Antigravity puede cerrar esta brecha.

6.1 La Skill unity-test-runner

Esta Skill orquesta el ciclo de pruebas:

1. **Ejecución:** Llama al wrapper de Python para ejecutar Unity.exe -runTests -testPlatform EditMode.
2. **Parsing de Resultados:** El wrapper localiza el archivo TestResults.xml (formato NUnit estándar) generado por el framework de pruebas.[40] Dado que Unity genera XML y los agentes prefieren JSON, el script de Python convierte este XML a un JSON simplificado que destaca fallos, mensajes de error y stack traces.[41]
3. **Análisis y Reparación:** El agente recibe el JSON. Si hay fallos, correlaciona el stack trace con el archivo fuente y el número de línea.
4. **Bucle de Refactorización:** El agente intenta corregir el código que causó el fallo y vuelve a ejecutar la prueba (respetando un límite de reintentos predefinido).[1]

6.2 Benchmarking de Rendimiento

Utilizando el paquete *Unity Performance Testing*, las pruebas pueden producir datos cuantitativos (grupos de muestreo, tiempos de ejecución, asignación de memoria). Un agente de Antigravity puede ser encargado de "Verificar regresiones de rendimiento".
El flujo sería:

1. Ejecutar pruebas de rendimiento vía CLI.[43]
2. Comparar el nuevo results.json contra un artefacto de línea base (baseline) almacenado en el historial del proyecto o en un sistema de control de versiones.
3. Alertar si algún sistema (e.g., PhysicsUpdate) ha incrementado su tiempo de ejecución más allá de un umbral tolerante (e.g., >10%).

Esto es vital para el desarrollo con DOTS, donde el rendimiento es la métrica principal de éxito.[43]

---

7. Seguridad y Gobernanza en Flujos de Trabajo Agénticos

Otorgar a un agente de IA la capacidad de ejecutar comandos de shell y modificar bases de código introduce riesgos de seguridad significativos que deben ser gestionados.

7.1 Perfil de Riesgo de la Herramienta run_command

La herramienta run_command es potente pero peligrosa. Un agente comprometido por "Prompt Injection" podría teóricamente ejecutar comandos destructivos (rm -rf /) o exfiltrar código fuente.[45]

7.2 Estrategias de Mitigación y Sandboxing

Antigravity emplea un modelo de "Humano en el Bucle" por defecto, solicitando aprobación

para comandos. Sin embargo, para tareas de alta frecuencia, esto genera fricción.
Estrategias de Gobernanza Recomendadas:

1. **Flag SafeToAutoRun:** Los autores de Skills pueden marcar comandos específicos y de bajo riesgo (como ls, git status, o la ejecución de scripts pre-aprobados) como seguros para ejecución automática. Esto se hace mediante anotaciones // turbo en el plan de implementación o flags específicos en la definición de la herramienta.[47]
2. **Listas de Permitidos (Allow-Lists):** La configuración corporativa de Antigravity debe restringir run_command para permitir solo la ejecución de scripts ubicados en rutas controladas (e.g., ./.agent/scripts/), bloqueando el acceso a rutas del sistema.[2]
3. **Contenerización Docker:** Para la ejecución de código no confiable o scripts con dependencias complejas (como entornos Python específicos para builds), el agente debe operar dentro de un contenedor Docker efímero. Esto asegura que incluso un comando malicioso afecte solo al contenedor y no al sistema operativo anfitrión.[49]

---

8. Conclusión

Google Antigravity representa una evolución crítica en las herramientas de desarrollo, transitando de la asistencia pasiva a la delegación activa. Para los desarrolladores de Unity 3D, esta plataforma ofrece una oportunidad única para automatizar el trabajo de "pegamento" que frecuentemente ralentiza la producción.
Al encapsular la compleja CLI de Unity, el parsing de logs y los patrones arquitectónicos (DOTS/ECS, VContainer) en **Skills de Antigravity** estandarizadas, los equipos pueden crear un "Artista Técnico Virtual" capaz de gestionar builds, forzar estilos de código y ejecutar pruebas de rendimiento de manera autónoma. La combinación del **Protocolo de Contexto de Modelo (MCP)** para el acceso profundo a datos y la **Divulgación Progresiva** para la gestión del contexto asegura que estos agentes permanezcan eficientes y precisos incluso en bases de código AAA masivas.
El futuro del desarrollo de videojuegos no reside solo en motores más rápidos, sino en pipelines más inteligentes. La implementación de los patrones arquitectónicos delineados en este informe —específicamente el wrapper de Python para Unity CLI y las definiciones de Skills basadas en metadatos— proporciona los cimientos para ese futuro.

Análisis Detallado de Componentes Clave

Tabla 1: Comparativa de Flujos de Trabajo Unity: Tradicional vs. Agéntico

Característica	Flujo Tradicional (Humano)	Flujo Agéntico Antigravity
Completado de Código	Usuario escribe, IDE sugiere snippets.	Agente planifica y escribe clases/sistemas enteros.
Refactorización	Renombrado manual, resolución de conflictos.	Agente analiza dependencias, refactoriza y verifica builds.

Proceso de Build	Usuario clica "Build", espera, revisa logs manualmente.	Agente ejecuta build en background, parsea logs, reporta errores JSON.
Testing	Usuario ejecuta Test Runner, interpreta XML/GUI.	Agente ejecuta tests, parsea resultados, identifica fallos, intenta fixes.
Contexto	Limitado a archivos abiertos en pestañas.	**MCP** permite acceso a BDs, docs y estructura completa del proyecto.
Arquitectura	Adherencia manual a patrones (DOTS/DI).	Skills fuerzan patrones (e.g., estructura ISystem) automáticamente.

Tabla 2: Estructura de Directorio de Skills Recomendada para Proyectos Unity

Ruta	Propósito	Tipo de Contenido
.agent/skills/unity-build/SKILL.md	Instrucciones para construir el player.	Markdown + YAML
.agent/skills/unity-build/scripts/build.py	Wrapper Python para Unity.exe -batchmode.	Python Script
.agent/skills/dots-gen/SKILL.md	Instrucciones para generar Sistemas ECS.	Markdown + YAML
.agent/skills/dots-gen/templates/System.cs.txt	Plantilla para sistemas Burst-compatibles.	Texto / C#
.agent/skills/log-parser/SKILL.md	Skill para leer/interpretar Editor.log.	Markdown + YAML
.agent/skills/log-parser/scripts/parse_log.py	Lógica Regex para extraer errores de logs.	Python Script

Guía de Implementación: La Skill "Build & Verify"

Para demostrar concretamente la investigación, a continuación se presenta la especificación arquitectónica para una Skill de "Construcción y Verificación", que constituye el "Hola Mundo" de la automatización agéntica en Unity.

1. Los Metadatos (Frontmatter de SKILL.md):

YAML

name: unity-build-verify
description: Ejecuta una compilación headless de Unity para la plataforma objetivo (Windows/Android), parsea los logs en busca de errores y reporta el estado. Úsese cuando el

usuario pida "construir el juego" o "verificar si el proyecto compila".

2. Las Instrucciones (Cuerpo Markdown):

- **Objetivo:** Ejecutar una build exitosa y devolver la ruta del ejecutable o la lista de errores.
- **Paso 1:** Identificar la plataforma de build solicitada. Por defecto StandaloneWindows64 si no se especifica.
- **Paso 2:** Construir el comando CLI. Usar la herramienta run_command para invocar el script embebido build.py.
    - *Restricción:* Debe usar -batchmode y -quit.
    - *Restricción:* Debe redirigir el logging a un archivo temporal o stdout para el parsing.
- **Paso 3:** Interpretar la salida JSON del script de Python.
    - Si exit_code == 0: Reportar éxito y ubicación del build.
    - Si exit_code!= 0: Resumir los 3 errores principales del log parseado y preguntar al usuario si desea que el agente intente arreglarlos.

3. La Capa de Ejecución (scripts/build.py): Este script actúa como capa de traducción. Encuentra el ejecutable del Editor de Unity (usando Unity Hub o variables de entorno), construye los argumentos y lanza el proceso. Crucialmente, monitorea el flujo de salida. Utiliza subprocess.Popen de Python para capturar stdout y stderr en tiempo real, alimentando esta información de vuelta al agente Antigravity para que el usuario pueda ver el progreso en la UI del Agent Manager.[16]

Al implementar esta arquitectura, los desarrolladores efectivamente "despegan" (usando la terminología de Google) de los detalles mundanos de la gestión de la CLI y operan al nivel de intención y verificación.

Obras citadas

1. Build with Google Antigravity, our new agentic development platform, fecha de acceso: enero 23, 2026, https://developers.googleblog.com/build-with-google-antigravity-our-new-agentic-development-platform/
2. Getting Started with Google Antigravity, fecha de acceso: enero 23, 2026, https://codelabs.developers.google.com/getting-started-google-antigravity
3. Introducing Google Antigravity, a New Era in AI-Assisted Software Development, fecha de acceso: enero 23, 2026, https://antigravity.google/blog/introducing-google-antigravity
4. How to Set Up and Use Google Antigravity - Codecademy, fecha de acceso: enero 23, 2026, https://www.codecademy.com/article/how-to-set-up-and-use-google-antigravity
5. Google Antigravity Documentation, fecha de acceso: enero 23, 2026, https://antigravity.google/docs/home
6. Documentation - Google Antigravity, fecha de acceso: enero 23, 2026,

https://antigravity.im/documentation
7. Tutorial : Getting Started with Google Antigravity Skills - Medium, fecha de acceso: enero 23, 2026, https://medium.com/google-cloud/tutorial-getting-started-with-antigravity-skills-864041811e0d
8. Spring AI Agentic Patterns (Part 1): Agent Skills - Modular, Reusable Capabilities, fecha de acceso: enero 23, 2026, https://spring.io/blog/2026/01/13/spring-ai-generic-agent-skills/
9. Agent Skills - Google Antigravity Documentation, fecha de acceso: enero 23, 2026, https://antigravity.google/docs/skills
10. Authoring Google Antigravity Skills, fecha de acceso: enero 23, 2026, https://codelabs.developers.google.com/getting-started-with-antigravity-skills
11. Agent Skills | Cursor Docs, fecha de acceso: enero 23, 2026, https://cursor.com/docs/context/skills
12. Use Agent Skills in VS Code, fecha de acceso: enero 23, 2026, https://code.visualstudio.com/docs/copilot/customization/agent-skills
13. skills/skills/skill-creator/SKILL.md at main · anthropics/skills - GitHub, fecha de acceso: enero 23, 2026, https://github.com/anthropics/skills/blob/main/skills/skill-creator/SKILL.md
14. Agent Skills - Claude Docs, fecha de acceso: enero 23, 2026, https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview
15. Python for Unity | 2.0.1-preview.2 - Unity - Manual, fecha de acceso: enero 23, 2026, https://docs.unity3d.com/Packages/com.unity.scripting.python@2.0/manual/index.html
16. Unity Build Automation with Python: Complete Multi-Platform Tutorial - Medium, fecha de acceso: enero 23, 2026, https://medium.com/@studio.angry.shark/unity-build-automation-with-python-complete-multi-platform-tutorial-60162c103028
17. Manual: Unity Editor command line arguments reference, fecha de acceso: enero 23, 2026, https://docs.unity3d.com/6000.3/Documentation/Manual/EditorCommandLineArguments.html
18. Using the Command Line Toolset to Run Unity Tests - KinematicSoup Technologies Inc., fecha de acceso: enero 23, 2026, https://kinematicsoup.com/news/using-the-command-line-toolset-to-run-unity-tests
19. Command line arguments - Unity Manual, fecha de acceso: enero 23, 2026, https://docs.unity.cn/ru/2021.1/Manual/CommandLineArguments.html
20. Automating Unity Builds — Part 1 : Unity via Command Line | by Fadhil Noer Afif | Medium, fecha de acceso: enero 23, 2026, https://fadhilnoer.medium.com/automating-unity-builds-part-1-ba0c60e8d06b
21. How to Render on Headless mode in Unity Linux? : r/Unity3D - Reddit, fecha de acceso: enero 23, 2026, https://www.reddit.com/r/Unity3D/comments/1liv41c/how_to_render_on_headless

_mode_in_unity_linux/
22. live output from subprocess command - python - Stack Overflow, fecha de acceso: enero 23, 2026, https://stackoverflow.com/questions/18421757/live-output-from-subprocess-command
23. Parse and Clean Log Files in Python - GeeksforGeeks, fecha de acceso: enero 23, 2026, https://www.geeksforgeeks.org/python/parse-and-clean-log-files-in-python/
24. Using DOTS to optimize GameObject gameplay: A case study from Survival Kids | Unite 2025 - YouTube, fecha de acceso: enero 23, 2026, https://www.youtube.com/watch?v=ZkvK0mX-id4
25. Unity Burst Compiler: Complete Performance Optimization Guide 2025, fecha de acceso: enero 23, 2026, https://generalistprogrammer.com/tutorials/unity-burst-compiler-complete-performance-optimization-guide
26. ISystem overview | Entities | 1.2.2-pre.1 - Unity - Manual, fecha de acceso: enero 23, 2026, https://docs.unity3d.com/Packages/com.unity.entities@1.2/manual/systems-isystem.html
27. Systems comparison | Entities | 1.0.16 - Unity - Manual, fecha de acceso: enero 23, 2026, https://docs.unity3d.com/Packages/com.unity.entities@1.0/manual/systems-comparison.html
28. Create the system for the spawner example | Entities | 1.3.14 - Unity - Manual, fecha de acceso: enero 23, 2026, https://docs.unity3d.com/Packages/com.unity.entities@1.3/manual/ecs-workflow-create-systems.html
29. Upgrading from Entities 0.51 to 1.0 - Unity - Manual, fecha de acceso: enero 23, 2026, https://docs.unity3d.com/Packages/com.unity.entities@1.0/manual/upgrade-guide.html
30. Comparing to Zenject | VContainer, fecha de acceso: enero 23, 2026, https://vcontainer.hadashikick.jp/comparing/comparing-to-zenject
31. Dependency Injection in Unity C# | by James Lafritz | Medium, fecha de acceso: enero 23, 2026, https://ktmarine1999.medium.com/dependency-injection-in-unity-c-29c3a352ea92
32. Hello World - VContainer, fecha de acceso: enero 23, 2026, https://vcontainer.hadashikick.jp/getting-started/hello-world
33. hadashiA/VContainer: The extra fast, minimum code size, GC-free DI (Dependency Injection) library running on Unity Game Engine. - GitHub, fecha de acceso: enero 23, 2026, https://github.com/hadashiA/VContainer
34. Injection on instantiated Scriptable Object from Game Object Context Installer, fecha de acceso: enero 23, 2026, https://stackoverflow.com/questions/77071614/injection-on-instantiated-scriptabl

e-object-from-game-object-context-installer
35. Antigravity Editor: MCP Integration, fecha de acceso: enero 23, 2026, https://antigravity.google/docs/mcp
36. Architecture overview - Model Context Protocol, fecha de acceso: enero 23, 2026, https://modelcontextprotocol.io/docs/learn/architecture
37. Connect Google Antigravity IDE to Google's Data Cloud services | Google Cloud Blog, fecha de acceso: enero 23, 2026, https://cloud.google.com/blog/products/data-analytics/connect-google-antigravity-ide-to-googles-data-cloud-services
38. Unity Game Dev Supercharger - Claude Code Skill - MCP Market, fecha de acceso: enero 23, 2026, https://mcpmarket.com/es/tools/skills/unity-game-dev-supercharger
39. Agentic Automation in Google Workspace: Bridging Antigravity and Gemini 3.0 - GitHub Gist, fecha de acceso: enero 23, 2026, https://gist.github.com/tanaikech/da7deaad19f75db0fd473e84e856529e
40. Run tests from the command line - Unity - Manual, fecha de acceso: enero 23, 2026, https://docs.unity3d.com/6000.3/Documentation/Manual/test-framework/run-tests-from-command-line.html
41. Recording test results with the Launchable CLI, fecha de acceso: enero 23, 2026, https://help.launchableinc.com/sending-data-to-launchable/using-the-launchable-cli/recording-test-results-with-the-launchable-cli/
42. Run cucumber by tag or run all tests - Stack Overflow, fecha de acceso: enero 23, 2026, https://stackoverflow.com/questions/77356810/run-cucumber-by-tag-or-run-all-tests
43. Performance benchmarking in Unity: How to get started, fecha de acceso: enero 23, 2026, https://unity.com/blog/engine-platform/performance-benchmarking-in-unity-how-to-get-started
44. Unity ECS Performance Testing: The Way To The Best Performance - gamedev.center, fecha de acceso: enero 23, 2026, https://gamedev.center/unity-ecs-performance-testing-the-way-to-the-best-performance/
45. Antigravity Grounded! Security Vulnerabilities in Google's Latest IDE - Embrace The Red, fecha de acceso: enero 23, 2026, https://embracethered.com/blog/posts/2025/security-keeps-google-antigravity-grounded/
46. How Code Execution Drives Key Risks in Agentic AI Systems | NVIDIA Technical Blog, fecha de acceso: enero 23, 2026, https://developer.nvidia.com/blog/how-code-execution-drives-key-risks-in-agentic-ai-systems/
47. Google's Antigravity IDE: The First AI That Tried to Hack My Local Env (Security Review), fecha de acceso: enero 23, 2026, https://www.reddit.com/r/AI_Agents/comments/1p3tvvs/googles_antigravity_ide_t

he_first_ai_that_tried/
48. system-prompts-and-models-of-ai-tools/Google/Antigravity/Fast Prompt.txt at main - GitHub, fecha de acceso: enero 23, 2026, https://github.com/x1xhlol/system-prompts-and-models-of-ai-tools/blob/main/Google/Antigravity/Fast%20Prompt.txt
49. Takedown: How It's Done in Modern Coding Agent Exploits - arXiv, fecha de acceso: enero 23, 2026, https://arxiv.org/html/2509.24240v1
50. (Analysis)Security of AI Agents. 1. What Problem Does the Paper, fecha de acceso: enero 23, 2026, https://medium.com/@trivajay259/analysis-security-of-ai-agents-eed70c44c3e0